

Evaluation - E 2

Intégrer des modèles et des services d'intelligence artificielle

Bloc de compétences 2. référence : REAC page 5-9. Rapport de 15 à 20 pages.

Sommaire

- **C6. Organiser et réaliser une veille technique et réglementaire** en animant le travail collectif de sélection des sources, de collecte, de traitement et de partage des informations afin de formuler des recommandations pour le projet toujours en phase avec l'état de l'art.
 - **C7. Identifier des services d'intelligence artificielle préexistants** à partir de l'expression de besoin en fonctionnalités d'intelligence artificielle, en réalisant un benchmark de services existants et en analysant leurs caractéristiques pour formaliser une ou plusieurs recommandations de services adaptés au besoin.
 - **C8. Paramétrer un service d'intelligence artificielle** en suivant sa documentation technique et en respectant les spécifications du projet, afin de permettre l'intégration des connecteurs du service dans le système d'information.
-
-

. Le Benchmark (C7) : À partir d'un besoin (réel ou fictif), vous devez comparer plusieurs services d'IA, justifier pourquoi vous en écarterez certains (coût, RGPD, éco-responsabilité), et préconiser le meilleur . Paramétrage (C8) : Expliquer l'installation, la configuration des droits d'accès et le monitoring du service retenu .

C6. Organiser et réaliser une veille technique et réglementaire

En animant le travail collectif de sélection des sources, de collecte, de traitement et de partage des informations afin de formuler des recommandations pour le projet toujours en phase avec l'état de l'art.

comment automatiser la veille (flux RSS, agrégateurs comme InoReader) pour rester à jour sur les réglementations et les modèles IA

Afin de garantir que les choix technologiques et réglementaires du projet **Drink Safe** soient toujours en phase avec l'état de l'art, j'ai mis en place un dispositif de veille automatisé, structuré et collaboratif :

1. Thématiques ciblées, la veille se concentre sur deux piliers essentiels au projet :

- **La veille réglementaire et sécurité :**

- Suivi des évolutions du **RGPD** (traitement des données personnelles)
- Suivi des évolutions de l'**IA Act** européen
- Suivi des évolutions des standards de cybersécurité liés aux APIs (OWASP).

- **La veille technologique IA :**

- Suivi des évolutions des modèles de Machine Learning (classification, scikit-learn, XGBoost)
- Suivi des services d'extraction de texte (OCR).

1. Outils d'automatisation et de collecte Pour éviter une recherche manuelle chronophage. Mise en place d'un agrégateur de flux RSS (Inoreader / Feedly) . Cet outil centralise automatiquement les actualités, et permet de gagner en efficacité et de ne manquer aucune information critique.

2. Identification et qualification des sources La fiabilité des informations est primordiale. Les sources intégrées à l'agrégateur ont été qualifiées selon des critères stricts (notoriété de l'auteur, absence de conflits d'intérêts, et récence de publication) :

- Sources réglementaires officielles : La [CNIL](#) (référence absolue pour le RGPD) et les publications de l'[ANSSI](#).
- Sources sécurité : Le blog de la fondation [OWASP](#).
- Sources techniques : Agrégateurs communautaires validés par les pairs [Hacker News](#), blogs d'ingénierie reconnus [Towards Data Science](#) et documentations officielles des frameworks (FastAPI, scikit-learn).

1. Organisation et planification La veille nécessite de la régularité. J'ai instauré une routine stricte en bloquant un créneau fixe d'une heure chaque semaine (le vendredi matin) pour lire, trier et traiter les flux remontés par l'agrégateur.

2. Synthèse, Partage et Accessibilité Une veille n'a de valeur que si elle est partagée. À l'issue de mon créneau hebdomadaire, je rédige de courtes synthèses des informations les plus pertinentes (ex: mise à jour critique d'une faille OWASP, ou nouveau modèle OCR). Ces synthèses sont partagées à l'équipe de développement et au Responsable d'Exploitation via un canal de communication asynchrone (Slack ou tableau Trello). Conformément aux engagements du projet, ces synthèses sont formatées pour respecter les recommandations d'accessibilité numérique (contrastes clairs, hiérarchie de l'information et textes alternatifs), en m'appuyant sur les recommandations de l'association [Valentin Haüy](#) et du standard [RGAA](#)

C7. Identifier des services d'intelligence artificielle préexistants

à partir de l'expression de besoin en fonctionnalités d'intelligence artificielle, en réalisant un benchmark de services existants et en analysant leurs caractéristiques pour formaliser une ou plusieurs recommandations de services adaptés au besoin.

Afin de répondre à la **User Story** de l'*agent de terrain* ("éviter la saisie manuelle en téléversant une fiche laboratoire"), j'ai dû identifier un service d'Intelligence Artificielle capable d'extraire le texte (OCR) depuis des documents non structurés (PDF, PNG, JPG).

OCR : Optical character recognition / Reconnaissance Optique de Caractères

Expression du besoin et contraintes du projet

Objectif fonctionnel : Extraire automatiquement les valeurs numériques de 9 mesures physico-chimiques depuis un rapport d'analyse d'eau.

Contraintes techniques : La solution doit s'intégrer facilement via une requête HTTP asynchrone à notre API métier (FastAPI), sans alourdir l'empreinte mémoire du serveur local.

Contraintes métiers : Budget limité (MVP) et respect du RGPD.

Benchmark des solutions existantes J'ai étudié trois approches distinctes pour répondre à ce besoin

Solution 1 - Tesseract OCR (Open Source & Local)

Adéquation : Bonne. Le traitement reste en local, ce qui est excellent pour le RGPD.

Contraintes techniques : Très forte. Nécessite d'installer des binaires lourds (libtesseract) dans le conteneur Docker de l'API, ce qui alourdit considérablement l'image et la consommation CPU. De plus, les performances sur des PDF mal scannés nécessitent beaucoup de pré-traitement (OpenCV).

Bilan : Écarté Surcoût de complexité d'infrastructure pour un simple MVP.

Solution 2 - Google Cloud Vision API (Service SaaS GAFAM)

Adéquation : Excellente.

La reconnaissance de texte est l'une des plus puissantes du marché.

Contraintes techniques et RGPD :

Nécessite la configuration complexe d'une Service Account Key (IAM) et l'enregistrement d'une carte bancaire. Les données sont envoyées sur des serveurs américains.

Bilan : Écarté.

Solution surdimensionnée (overkill), complexe à configurer pour le commanditaire, et posant des défis potentiels de souveraineté des données.

Solution 3 - OCR.space (API REST Freemium)

Adéquation : Très bonne. Le service prend en charge nativement les PDF et les images, et renvoie un fichier JSON brut facile à traiter (via des expressions régulières) directement dans FastAPI.

Éco-responsabilité : Mutualise la puissance de calcul sur un service Cloud spécialisé et optimisé pour l'OCR permet de réduire la consommation énergétique de nos propres serveurs (démarche Green IT locale).

Contraintes techniques : Dépendance à la disponibilité du réseau externe.

Conclusion et Préconisation À l'issue de cette analyse

Je préconise l'intégration du service [OCR.space](#)

C'est la solution la plus pragmatique :

- offre un tier gratuit (Free Tier) suffisant pour le volume attendu par la collectivité
- s'intègre nativement à notre API FastAPI via la librairie `requests`
- n'alourdit pas notre infrastructure Docker.

Pour pallier la contrainte de dépendance réseau, je prévois d'implémenter un mécanisme de secours [fallback gracieux](#) dans le code pour gérer les potentiels temps d'arrêt de l'API externe.

C8. Paramétrer un service d'intelligence artificielle

en suivant sa documentation technique et en respectant les spécifications du projet, afin de permettre l'intégration des connecteurs du service dans le système d'information.

Afin d'intégrer le service préconisé (OCR.space) dans la plateforme Drink Safe, j'ai procédé à son paramétrage technique et à sa sécurisation au sein de notre API FastAPI.

Création de l'environnement et gestion des accès

J'ai créé un compte SaaS sur le tier gratuit d'[OCR.space](#) afin de générer une clé d'authentification unique. Pour des raisons de sécurité, cette clé n'est jamais écrite en dur dans le code source. Elle est stockée de manière sécurisée dans les variables d'environnement du serveur (fichier `.env`) et chargée dynamiquement au démarrage du conteneur Docker.

Paramétrage des dépendances et interconnexion

Le sous-module `src/routes/ocr.py` de notre API communique avec le service via la librairie standard Python `requests`.

Le paramétrage de l'API tierce a été optimisé pour le besoin métier en suivant la [documentation officielle de l'API OCR.space](#): `language='fre'` : Optimisation du

moteur de reconnaissance pour le français. `isTable=True` : Activation du parsing en tableau pour mieux respecter l'alignement des colonnes des fiches laboratoires. `isOverlayRequired=False` : Désactivation des coordonnées spatiales du texte (Bounding Boxes) pour alléger le poids du payload JSON retourné et économiser de la bande passante. `scale=True` : Amélioration automatique de la résolution et de la lisibilité, particulièrement utile pour les fiches laboratoires scannées au format PDF.

[waterflow/src/routes/ocr.py](#) :

```
payload_ocr: Dict[str, Any] = {
    "apikey": ocr_api_key,
    "language": "fre",          # Gestion des accents
    français
    "isTable": "true",
    # Maintien de l'alignement des valeurs
    "isOverlayRequired": "false", # Optimisation réseau et
    RAM
    "scale": "true"            # Amélioration de la
    lecture
}
```

Traitement des données et Parsing L'API OCR.space

renvoie un JSON contenant un bloc de texte brut monolithique (ParsedText). J'ai paramétré une logique d'extraction spécifique côté serveur (via des expressions régulières - Regex) pour scanner ce texte brut, isoler les 9 noms de molécules, et extraire les valeurs numériques associées afin de les typer en float via Pydantic.

Monitoring et Résilience du service

L'intégration d'un service externe introduit un risque de défaillance réseau. J'ai donc configuré un monitoring spécifique (intégré à la supervision globale de l'application) :

- **Supervision des échecs** : Chaque erreur de l'API OCR (Timeout, dépassement de quota) incrémente automatiquement une métrique Prometheus spécifique `ocr_failures_total`.
- **Fallback (Dégradation gracieuse)** : Si le service est hors ligne, un bloc `try/except` intercepte l'erreur.

Au lieu de faire crasher le conteneur API, l'application renvoie un statut HTTP 200 (pending) à l'interface client qui affiche un message jaune d'avertissement indiquant que le fichier est mis en attente.